

CHAPTER 05

Connecting Clients and Servers



LEARN



BUILD

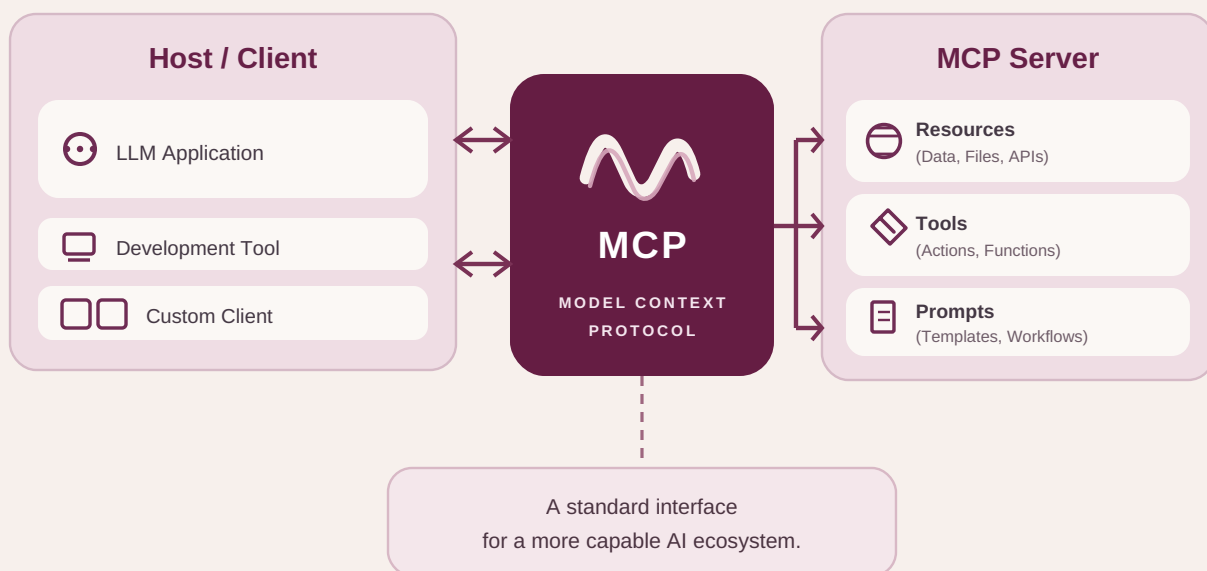


INTEGRATE



EXTEND

Installing Claude Desktop and wiring it to local and remote MCP servers via connectors and config files.



Premium Revision Notes

CONCEPTS

Clear Explanations

IMPLEMENTATION

Practical Code

DIAGRAMS

Visual Learning

INTERVIEW PREP

Key Questions

Connecting Clients and Servers

The *Why* established the problem MCP solves, and the *What* covered its architecture, primitives, and lifecycle. This chapter begins the **How**: the practical work of wiring an MCP host up to real servers. Everything here happens without writing a single line of server code — the goal is to build an accurate mental model of how a host discovers, authorizes, and talks to a server before Chapter 6 covers building a server from scratch.

All communication in MCP happens between a client and a server. This chapter puts that communication in front of you directly, using a ready-made host (Claude Desktop) and a mix of ready-made local and remote servers. Two servers will connect through a built-in **connector** with a single click, and two will connect by hand-editing a **JSON configuration file** — deliberately covering every combination of local/remote × connector/config so the trade-offs between the two connection methods are concrete rather than theoretical.

1. Two Ways to Connect a Server to a Host

Every MCP-capable host exposes exactly two mechanisms for attaching a server: editing a configuration file directly, or clicking through a built-in connector. Both end at the same place — a running client-server connection — but they differ in who did the engineering work to get there.

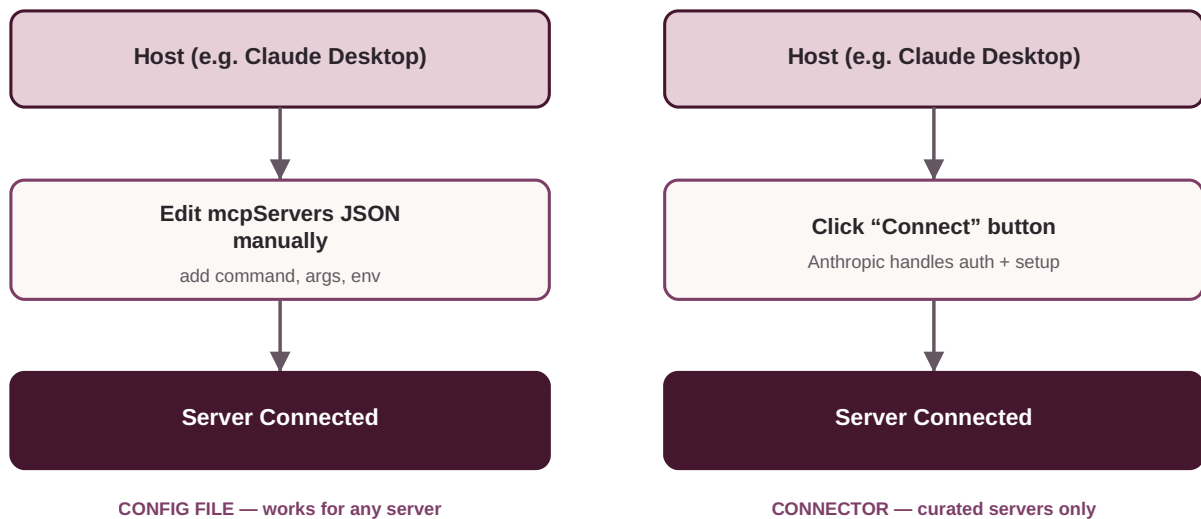


Figure 5.1 — The two connection paths converge on the same outcome: a connected MCP server, reachable via its tools, resources, and prompts.

1.1 The Configuration File Path

Every MCP host that supports the config-file approach stores a JSON file (for Claude Desktop, reached via **Settings** → **Developer** → **Edit Config**) with a top-level `mcpServers` object. Each entry names a server and tells the host how to start it:

```
{
  "mcpServers": {
    "server-name": {
      "command": "<absolute path to an executable>",
      "args": ["<script or package identifier>"],
      "env": { "SOME_KEY": "<YOUR_API_KEY>" }
    }
  }
}
```

The host reads this file on startup, launches `command` with the given `args` (and any `env` variables) for every entry under `mcpServers`, and establishes an MCP session with each resulting process. This is a manual but universal mechanism — it works for any server, official or independently built, local or remote, with no gatekeeping.

1.2 The Connector Path

A **connector** is a built-in feature that links a host to an MCP server automatically, without the user ever opening a configuration file. Clicking a single button handles authentication, credential storage, and the underlying connection setup behind the scenes.

IMPORTANT

Connectors exist primarily to serve non-technical users. Most people using an AI host simply want to link it to tools they already use — Google Drive, Notion, Slack — without locating a server's setup instructions, copying configuration snippets, or troubleshooting a malformed JSON file. A connector collapses that entire process into one click, and because Anthropic's own team writes and maintains the connector code, it is also more consistent and secure than an ad hoc manual setup.

2. Why Config Files Still Matter

If connectors are safer and easier, an obvious question follows: why not force every MCP server to go through a connector? Two structural reasons prevent that.

REASON 1 — CONNECTORS DO NOT SCALE

A connector is not free to build. Anthropic's team has to write the integration, host it, handle OAuth flows, apply security patches, manage rate limits, and keep the experience stable over time. That labor is reasonable for a small set of extremely popular SaaS tools, but thousands of MCP servers already exist, with new ones appearing daily. No single team can hand-build and maintain a connector for every one of them.

REASON 2 — MCP IS AN OPEN STANDARD

MCP was designed from day one so that anyone can build a client and anyone can build a server, with no central approval step. If every server had to wait for Anthropic to build and ship a matching connector before it could connect to a host like Claude Desktop, that openness would collapse into a gatekept ecosystem — one company deciding which servers are allowed to run. The config-file path exists precisely so a server built today can be connected today, with no review or approval required.

Property	Configuration File	Connector
Setup effort	Manual JSON editing	One click
Server coverage	Any MCP server	Only curated, popular SaaS tools
Auth & credential handling	User's responsibility	Handled behind the scenes
Who maintains it	The server author / the user	Anthropic's team
Time-to-connect for a brand-new server	Immediate	Only after a connector is built

🔄 Quick Revision

- A host connects to an MCP server either by editing a JSON `mcpServers` config file, or through a built-in connector.
- Config files are manual but universal; connectors are one-click but limited to servers Anthropic has built an integration for.
- Connectors exist to remove technical friction for non-technical users adopting popular SaaS tools.
- Config files exist because connectors do not scale to thousands of servers, and because MCP's value as an *open* standard depends on servers not needing pre-approval to connect.

3. Local vs. Remote Servers

Independent of how a server is connected, every MCP server is also either **local** or **remote**, based on where its process actually runs.

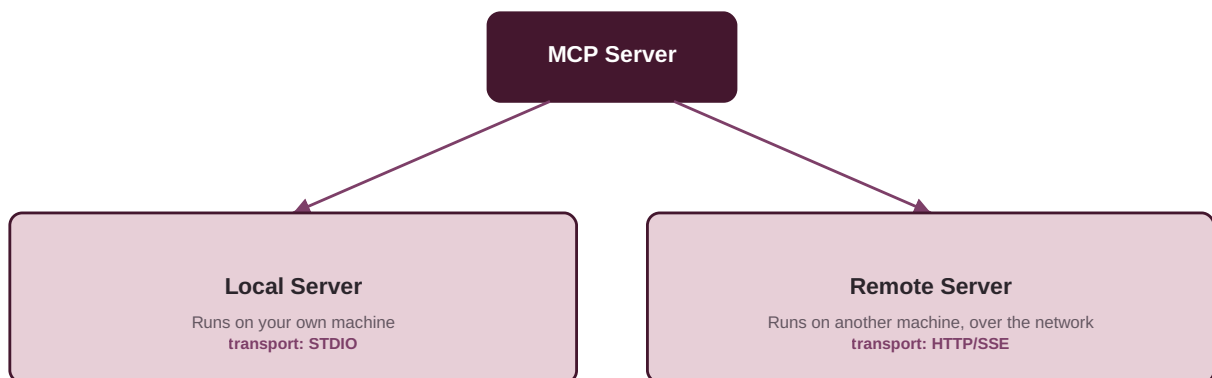


Figure 5.2 — A local server is a program the host launches on your own machine; a remote server already exists elsewhere and is reached over the network.

A **local server** is a program that runs on your own machine. The host is responsible for starting it (usually as a subprocess) and talks to it over **STDIO**. The filesystem server and the Manim server used later in this chapter are both local: their code is either bundled with the host or cloned onto your machine before use.

A **remote server** already exists on a machine somewhere on the internet. Nothing needs to be installed locally to run it — the host simply connects to it over **HTTP/SSE**. Google Drive's server is remote in this sense: Google runs it, and Claude Desktop is only a client reaching out to it.

COMMON MISTAKE

A server's setup instructions can look remote at a glance and still be local underneath. If a configuration entry's **command** is something like `npx` or `uvx`, look closely at what that command actually does: `npx` installs and runs a package directly on your machine, which makes the resulting server *local*, even though no repository was manually cloned. A server is remote only when its **command/args** point at code that executes on someone else's infrastructure, not when the install step merely feels lightweight.

🔄 Quick Revision

- Local servers run as a process on your own machine, launched by the host, and communicate over STDIO.
- Remote servers run on infrastructure elsewhere and are reached over HTTP/SSE.
- A one-line install command (like `npx` or `uvx`) does not by itself make a server remote — check what code actually executes and where.

4. Setting Up the Host: Claude Desktop

This chapter uses Claude Desktop as the host because it ships with both connection mechanisms available side by side. Any MCP-compatible host would demonstrate the same underlying protocol behavior.

4.1 Installation

Download Claude Desktop for your operating system and sign in. The interface is a familiar chat window — the MCP-specific surface to notice is the **Search and Tools** option, which opens a panel listing every server currently reachable from the host: some offered directly (Google Drive, Gmail, Calendar), and an **Add Connectors** option that opens a broader catalog of connector-ready servers grouped as desktop extensions (local servers) and web integrations (remote servers).

BEST PRACTICE

Whenever a new MCP server is added — through either connection method — the host must be fully closed and reopened before the new server appears in the tools list. The host only reads its configuration and re-establishes connector sessions on startup.

5. Local Server via Connector: the Filesystem Server

The filesystem MCP server gives a host the ability to read, write, and organize files in specific directories on your machine. It is one of the servers with a ready-made connector, so setup requires no configuration file at all.

5.1 Setup

Search and Tools

Add Connectors

Desktop Extensions → Filesystem

Install

After installing, the server does not automatically gain access to the entire machine. By default it has access to *no* directory until you explicitly grant one — for example, adding only the Desktop folder means the server can read and create files there, and nowhere else. Additional directories (Downloads, a specific project folder) can be added the same way. Once a directory is chosen, the connector is enabled and the host is restarted.

IMPORTANT

Directory scoping is a deliberate safety boundary, not an inconvenience. Granting a server access to an entire machine means any unexpected behavior in that server — a bug, a bad instruction, a compromised update — can touch every file you own. Scoping access to only the directories a task genuinely needs limits the blast radius of anything going wrong.

5.2 What Happens When a Tool Is Called

Once connected, asking the host something like *"are there any PDF files on my Desktop?"* triggers the sequence every MCP tool call follows: the host recognizes that a connected server exposes a matching tool, and — critically — asks for the user's permission before running it, every single time a tool is invoked.

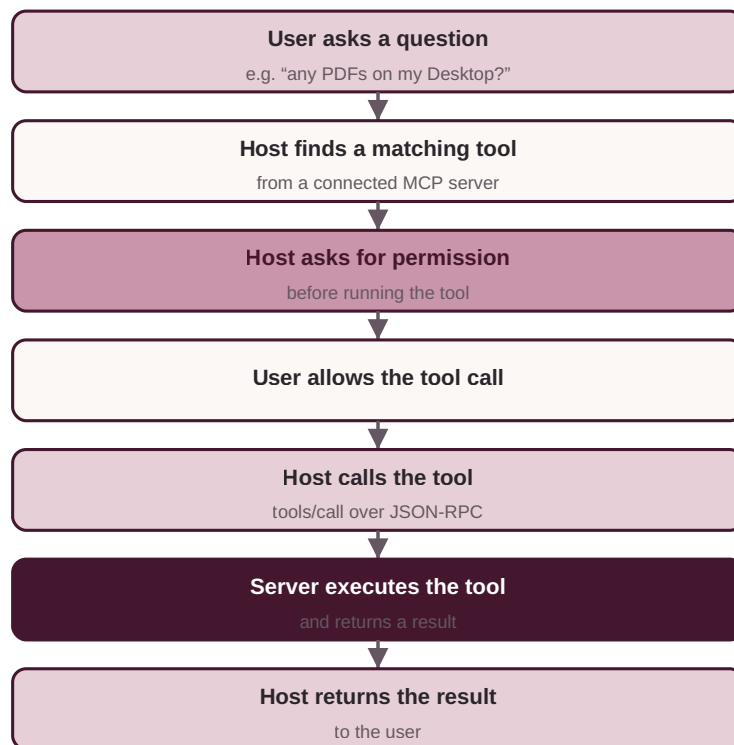


Figure 5.3 — A host never calls a connected tool silently; the user is asked to allow each invocation before it runs.

This permission step is not a one-time setup formality — it repeats for every tool call, which is what makes MCP's tool-use model safe for actions with real side effects (writing a file, posting a tweet, modifying a spreadsheet). Beyond simple reads, the filesystem server can also create new files: for example, asking the host to write a short Python script and save

it to the Desktop results in the host generating the code and then invoking the server's write tool, again behind an explicit permission prompt.

REAL WORLD INSIGHT

The filesystem server is commonly used for tasks beyond simple reads and writes — reorganizing a cluttered downloads folder, or summarizing the contents of an entire project directory. The tool primitives stay the same (list, read, write); what changes is the instruction given to the host.

🔄 Quick Revision

- The filesystem server is local and connector-based — no manual configuration is required.
- Access is scoped to explicitly chosen directories; nothing is accessible by default.
- Every tool call — read or write — requires explicit user permission at the moment it happens, not just at setup.

6. Local Server via Config File: the Manim Server

Manim is a Python animation library used to build precise, code-driven mathematical visualizations (the kind popularized by explainer-style math channels). Writing raw Manim code by hand for a specific animation is notoriously time-consuming; an MCP server built around Manim lets a host generate that code from a plain-English description and render the resulting video — and because no official connector exists for it, this is the config-file path end to end.

6.1 Prerequisites

- Python installed on the machine.
- The `manim` package: `pip install manim`
- The `mcp` package: `pip install mcp`
- The server's source repository, cloned locally (this is a local server, so its code has to physically exist on your machine before the host can launch it).

6.2 Locating the Required Paths

The config entry for a local Python-based server needs two absolute paths: the Python interpreter, and the server's entry-point script.

FIND PYTHON'S PATH

```
where python
```

FIND THE CLONED REPO'S PATH

```
cd manim-mcp-server\src
cd
```

6.3 Editing the Configuration

With both paths in hand, add an entry to the host's `mcpServers` object (**Settings** → **Developer** → **Edit Config**):

```
{
  "mcpServers": {
    "manim-server": {
      "command": "<ABSOLUTE_PATH_TO_PYTHON>/python",
      "args": ["<ABSOLUTE_PATH_TO_REPO>/src/manim_server.py"],
      "env": {
        "MANIM_EXECUTABLE": "<ABSOLUTE_PATH_TO_MANIM_EXECUTABLE>"
      }
    }
  }
}
```

The `env` block pointing `MANIM_EXECUTABLE` at an absolute path is the more robust way to tell the server where to find the Manim renderer — it avoids depending on the executable happening to be discoverable on the system's `PATH` when the host launches the server as a subprocess.

COMMON MISTAKE

Pasting a config snippet into the wrong place in the JSON file is an easy way to break the whole file. New server entries must be added *inside* the existing `mcpServers` object, separated from the previous entry by a comma — not appended after the file's closing brace.

Save the file, close it, and fully restart the host, exactly as with any newly added server.

6.4 Testing the Server

A prompt describing a linear algebra visualization — a 2D coordinate grid, two basis vectors, a matrix transformation bending the grid, and the resulting transformed vectors — is enough for the host to generate the corresponding Manim code, execute it through the server, and return a rendered video.

COMMON MISTAKE

If LaTeX is not installed on the machine, mathematical symbols in a rendered animation may not display correctly; the server will typically fall back to an alternate rendering path. A full LaTeX installation resolves this but is a large download (roughly 10–15 GB), so it is worth treating as optional unless polished mathematical typesetting is required.

🔄 Quick Revision

- The Manim server is local and has no connector, so it is set up entirely through the config file.
- Local Python-based servers need two absolute paths in their config: the Python interpreter and the entry-point script.
- An `env` block is the reliable way to point a server at a required external executable (e.g. `MANIM_EXECUTABLE`).
- Missing LaTeX degrades math rendering quality but does not stop the server from producing a video.

7. Remote Server via Connector: Google Drive

Google Drive is one of the servers Anthropic has built a direct connector for, making it the simplest connection in this chapter.

Search and Tools

Drive Search

Sign in with Google

Connected

Once authenticated, the host can pull content directly from a Drive document into a conversation — summarizing a stored document, for instance, without the file ever being manually downloaded or pasted in.

IMPORTANT

This particular Google Drive server is **read-only**. The host can read and summarize existing documents, but it cannot create new files or edit existing ones. Read/write access is a property of what a given server's tools expose, not something the protocol grants automatically once a connection exists.

8. Remote Server via Config File: X (Twitter) and Weather

Two more servers round out the local/remote × connector/config matrix, and together they illustrate why checking a server's actual **command** matters more than trusting first impressions.

Server	Connection method	Local or remote	Why
Filesystem	Connector	Local	Bundled desktop extension, runs on your machine
Manim	Config file	Local	Repository cloned and run locally with Python
Google Drive	Connector	Remote	Runs entirely on Google's infrastructure
X (Twitter)	Config file	Local (initially assumed remote)	command is <code>npx</code> , which installs and runs the server on your machine
Weather	Config file	Remote	command is <code>uvx</code> pointing at a package that is fetched and run from outside the machine's own codebase

8.1 X (Twitter) Server

The X server's setup instructions provide a config snippet using `npx` as the launch command:

```
{
  "mcpServers": {
    "twitter": {
      "command": "npx",
      "args": ["-y", "<TWITTER_MCP_PACKAGE>"],
      "env": {
        "API_KEY": "<YOUR_API_KEY>",
        "API_SECRET": "<YOUR_API_KEY_SECRET>",
        "ACCESS_TOKEN": "<YOUR_ACCESS_TOKEN>",
        "ACCESS_TOKEN_SECRET": "<YOUR_ACCESS_TOKEN_SECRET>"
      }
    }
  }
}
```

The four credential values come from a developer account's Keys and Tokens page, and each must be individually regenerated and copied in. Despite the setup instructions initially reading as if this were a remote integration, `npx` is what gives it away as local: the command installs the package via `npm` and runs it as a subprocess on the machine itself, exactly like the Python-based servers above.

COMMON MISTAKE

A developer account's API access defaults to **read-only**. Posting a tweet through the server will fail with an authentication error until write permission is explicitly enabled in the developer portal's user-authentication settings, and the access tokens are regenerated afterward — a stale read-only token will not gain write access on its own.

8.2 Weather Server (True Remote)

The weather server's configuration also uses a single-command launcher, but here the distinction matters: the command is `uvx`, and the package it runs is fetched from a location outside the local codebase every time it starts, rather than an installed local repository.

```
{
  "mcpServers": {
    "weather": {
      "command": "<ABSOLUTE_PATH_TO_UVX>",
      "args": ["<PACKAGE_OR_GIT_REFERENCE>"],
      "env": {
        "WEATHER_API_KEY": "<YOUR_API_KEY>"
      }
    }
  }
}
```

The API key comes from a weather-data provider's free developer tier. Once connected, asking the host for current conditions in a given city routes the request through this server's tool, which calls the underlying weather API and returns a formatted result.

BEST PRACTICE

When a server call fails, treat the host's own diagnostic behavior as a first debugging signal. A capable host will often attempt a reasonable fallback (such as a general web search) and describe what went wrong — that description is usually enough to tell whether the issue is a bad credential, a network problem, or a misconfigured path, before diving into manual debugging.

🔄 Quick Revision

- A server's true local/remote status is determined by what its `command` actually executes, not by how its setup instructions read.
- `npm`-launched servers run locally via Node's package runner, even with no manual clone step.
- `uvx` can launch either a local or a genuinely remote-fetched package — check what it points at.
- Read and write access are independent permissions a developer account (or a server) may grant separately.

9. Discovering New MCP Servers

With thousands of MCP servers already published and more appearing constantly, a curated, categorized directory is the most reliable way to find one for a given need. Community-maintained "Awesome MCP Servers" style lists on GitHub collect servers by category — the filesystem, Manim, and other servers used in this chapter can typically be located through exactly this kind of listing, alongside dedicated MCP marketplaces and listing sites that serve the same purpose.

REAL WORLD INSIGHT

A productive way to explore this space is to pick one recurring, tedious workflow — drafting a newsletter from scattered research, triaging a cluttered downloads folder — and search specifically for a server that automates a piece of it, rather than browsing the full category list without a goal in mind.

10. Key Takeaways

- A host connects to an MCP server through exactly two mechanisms: a manually edited JSON config file, or a built-in connector.
- Connectors trade flexibility for ease of use; config files trade ease of use for universal coverage and zero gatekeeping.
- Config files remain essential because connector-building does not scale to the full MCP ecosystem, and because MCP's identity as an open standard depends on servers not needing pre-approval to connect.
- Every MCP server is independently either local (runs on your machine, `STDIO`) or remote (runs elsewhere, `HTTP/SSE`) — a factor separate from how it was connected.
- A launch command like `npm` or `uvx` does not automatically make a server remote; what matters is where the executed code actually runs.
- Every tool call a host makes against a connected server is gated behind an explicit, per-call user permission prompt.

- Directory- and permission-scoping (as with the filesystem server) limits the impact of anything going wrong with a connected server.

11. Revision Sheet

A compact reference for the four servers connected in this chapter.

Filesystem

Local · Connector

Scoped directory read/write, permission-gated per call.

Manim

Local · Config file

Generates and renders math visualizations from plain English.

Google Drive

Remote · Connector

Read-only document access via OAuth sign-in.

X (Twitter)

Local (via npX) · Config file

Read/write scoped separately in the developer portal.

12. Interview Questions

Q1. What are the two ways a host can connect to an MCP server, and what does each trade off?

Q2. Why can't every MCP server simply be shipped with an official connector?

Q3. What determines whether a server is local or remote, and why can a config's launch command be misleading about this?

Q4. Why does a host ask for permission before every single tool call, rather than once per session?

Q5. What is the purpose of scoping a filesystem server's access to specific directories rather than the whole machine?

13. Further Reading

- Official Model Context Protocol documentation and specification
- Community-maintained directories of publicly available MCP servers
- Claude Desktop's built-in connector catalog (Search and Tools → Add Connectors)